

LeafScan - Plant Disease Detection System

A Mini Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**

BY

Priyanshi Bijghawane, EN23CS301789

Priyanshi Mahajan, EN23CS301790

Radhika Gurjar, EN23CS301813

Under the Guidance of

Prof. Swati Tahiliani

Prof. Laxmi Kag



**MEDICAPS
UNIVERSITY**

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

Jan-June 202

Report Approval

The project work “**LeafScan Plant Disease Detection System**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Prof. Swati Tahiliani

Assistant Professor

Medicaps University

Internal Examiner

Prof. Laxmi Kag

Assistant Professor

Medicaps University

Declaration

I/We hereby declare that the project entitled “**Plant Disease Detection System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, completed under the supervision of **Prof. Swati Tahlilani & Prof. Laxmi Kag**, Faculty of Engineering, Mediacaps University Indore, is an authentic work.

Further, I/we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the students

Priyanshi Bijghawane, EN23CS301789

Priyanshi Mahajan, EN23CS301790

Radhika Gurjar, EN23CS301813

Certificate

We, **Prof. Swati Tahiliani & Prof. Laxmi Kag** certify that the project entitled **“Plant Disease Detection System”** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Priyanshi Bijghawane, EN23CS301789, Priyanshi Mahajan, EN23CS301790 & Radhika Gurjar, EN23CS30813** is the record carried out by him/them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Swati Tahiliani

Prof. Laxmi Kag

Faculty of Engineering

Medicaps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean, Faculty of Computing Engineering, Mediacaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Priyanshi Bijghawane, EN23CS301789

Priyanshi Mahajan, EN23CS301790

Radhika Gurjar, EN23CS301813

B.Tech. III Year (N)

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Abstract

The rapid growth in agriculture and the increasing demand for high-quality crop production have highlighted the importance of early and accurate detection of plant diseases. Traditional methods of disease identification rely heavily on manual inspection by experts, which can be time-consuming, costly, and prone to human error. Additionally, farmers may lack access to proper diagnostic tools, leading to delayed treatment and reduced crop yield. To address this problem, the proposed system, *Plant Disease Detection using Leaf Images*, aims to provide an efficient and automated solution for identifying plant diseases at an early stage.

The developed system is a web-based application that allows users to upload images of plant leaves for analysis. The application uses image processing and machine learning techniques implemented in Python to detect and classify diseases based on visual symptoms present on the leaves. The input image is processed and analyzed using a trained model, which compares it with a dataset of healthy and diseased leaf images stored in the database.

The system interprets the results and presents them in a simple and user-friendly format, making it easier for farmers and users to understand the type of disease and its severity. Additionally, the application provides information about possible causes and suggests preventive measures or treatments to control the disease, helping users take timely action and improve crop productivity.

The application is developed using modern web technologies, including HTML, CSS, and JavaScript for the frontend, while Python is used for backend processing and machine learning implementation. A database is used to store image data, disease information, and analysis results. The system follows a client-server architecture and is designed to be accessible across various devices such as desktops and smartphones through web browsers.

The proposed solution focuses on accuracy, usability, and accessibility, ensuring that even users with minimal technical knowledge can easily interact with the system. By integrating technology with agriculture, the system helps bridge the gap between traditional farming practices and modern intelligent solutions, promoting better crop management and sustainable farming.

Table of Contents

Serial No.	Content	Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction	1-4
	1.1 Introduction	1
	1.2 Literature Review	1-2
	1.3 Objectives	2-3
	1.4 Significance	3-4
	1.5 Research Design	4
Chapter 2	REQUIREMENTS SPECIFICATION	5-11
	2.1 User Characteristics	5-6
	2.2 Functional Requirements	6-7
	2.3 Dependencies	7-8
	2.4 Performance Requirements	8-9
	2.5 Hardware Requirements	9-10
	2.6 Software Requirements	10-11
Chapter 3	DESIGN	12-23
	3.1 Algorithm	12-13
	3.2 Function Oriented Design for procedural approach	13-15
	3.3 System Design	15
	3.3.1 Data Flow Diagrams	16
	3.3.2 Activity Diagram	17
	3.3.3 Flow Chart	18
	3.3.4 Class Diagram	19
	3.3.5 ER Diagram	20
	3.3.6 Sequence diagram	21
	3.4 Database Design	21

	3.4.1 Logical Database Design	21-23
Chapter 4	Implementation, Testing, and Maintenance	24-31
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	24-26
	4.2 Testing Techniques and Test Plans (According to project)	26-27
	4.3 Installation Instructions	28-30
	4.4 End User Instructions	30-31
Chapter 5	Results and Discussions	32-
	5.1 User Interface Representation	32-33
	5.2 Brief Description of Various Modules of the system	34-36
	5.3 Snapshots of system with brief detail of each	36-40
	5.4 Back Ends Representation	41
	5.5 Snapshots of Database Tables with brief description	41
Chapter 6	Summary and Conclusions	42
Chapter 7	Future scope	43
	Appendix	43
	Bibliography	44

List of Figures

Figure No.	Figure DESCRIPTION	Page No.
3.3.1	Data Flow Diagram	16
3.3.2	Activity Diagram	17
3.3.3	Flowchart	18
3.3.4	Class Diagram	19
3.3.5	ER Diagram	20
3.3.6	Sequence Diagram	21
5.1	Home Page	34
5.2	Image Upload Interface	35
5.3	Analysis Result Screen	36
5.4	How It Work	36
5.5	Why LeafScan	37
5.6	Back End Representation	41
5.7	Database Table	41

List of Tables

Table No.	Table Description	Page No.
4.1	Test Cases for LeafScan Plant Disease Detection	27

Abbreviations

The following abbreviations are used throughout this report:

Abbreviation	Full Form
ML	Machine Learning
AI	Artificial Intelligence
UI	User Interface
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DB	Database
SRS	Software Requirements Specification
UML	Unified Modeling Language
JPG	Joint Photographic Experts Group
PNG	Portable Network Graphics
OS	Operating System
RAM	Random Access Memory

Notations & Symbols

1. Technical Notations

Notation / Symbol	Description
Image Input	Leaf image uploaded by the user
Image Processing	Method used to prepare image
API	Communication between frontend and backend
JSON	Data format for results
AI Model	System used to detect plant diseases
UI	User Interface
UX	User Experience

2. System & Architecture Terms

Notation / Symbol	Description
Frontend	React.js application
Backend	Flask (Python) server
Model	Prediction system
Database	Stores results
Storage	Stores images
Cache	Temporary storage

3. External Services

Notation / Symbol	Description
Flask API	Handles requests
OpenAI API	AI assistant
Cloudinary	Image storage
Speech API	Voice feature

4. Diagram Symbols

Symbol	Meaning
Rectangle	Process
Circle / Oval	Start/End
Diamond	Decision
Arrow	Flow
Parallel Lines	Database
User	User

Chapter 1: Introduction

1.1 Introduction

In today's rapidly evolving agricultural landscape, ensuring crop health and maximizing yield have become critical challenges. Plant diseases significantly affect agricultural productivity, leading to economic losses and reduced food quality. Early and accurate detection of plant diseases is essential for effective crop management; however, traditional methods of disease identification rely heavily on manual inspection by experts, which can be time-consuming, costly, and prone to human error.

With the advancement of technology, image processing and machine learning techniques have emerged as powerful tools for automating plant disease detection. Farmers and agricultural practitioners often lack access to expert knowledge, especially in rural or remote areas, making it difficult to identify diseases at an early stage. This creates a gap between agricultural knowledge and its practical application in the field.

To address this issue, the project titled “**Plant Disease Detection Using Leaf Images**” is proposed. The system is designed to automatically detect and classify plant diseases by analyzing images of plant leaves. The application allows users to upload or capture leaf images, which are then processed using image preprocessing techniques and machine learning or deep learning algorithms to identify the presence of diseases.

Once the image is analyzed, the system extracts relevant features such as color, texture, and patterns associated with different plant diseases. Based on these features, the model predicts the type of disease affecting the plant and provides relevant information, including possible causes and recommended preventive measures or treatments. This helps users take timely action to reduce crop damage.

The system is developed as a user-friendly application, ensuring accessibility for farmers, researchers, and agricultural professionals. It can be deployed as a web-based or mobile-based platform, enabling usage across various devices such as smartphones, tablets, and computers. The primary objective of this project is to provide an efficient, accurate, and cost-effective solution for plant disease detection, thereby improving agricultural productivity and supporting sustainable farming practices.

1.2 Literature Review

Recent advancements in the fields of Computer Vision, Machine Learning, and Deep Learning have significantly contributed to the development of automated plant disease detection systems. The increasing availability of image datasets and improvements in image processing techniques have enabled researchers to accurately identify plant diseases from leaf images, thereby supporting early diagnosis and improving agricultural productivity.

Kumar et al. (2022) proposed a deep learning-based approach using Convolutional Neural Networks (CNNs) for plant disease classification. Their model demonstrated high accuracy in

identifying multiple plant diseases from leaf images by automatically extracting relevant features. This work highlights the effectiveness of CNN architectures in handling complex visual patterns such as texture, color variations, and disease spots present in infected leaves.

Naseer et al. (2024) conducted a comprehensive review of image-based plant disease detection techniques and categorized them into traditional image processing methods and deep learning approaches. Their study emphasized the importance of preprocessing steps such as image resizing, noise removal, and segmentation for improving model performance. The review also discussed the superiority of deep learning models over conventional methods in terms of accuracy and scalability.

Lei et al. (2018) explored the application of data mining and machine learning techniques in agriculture, focusing on the relationship between environmental factors and plant diseases. Their research utilized structured datasets and classification algorithms to predict disease occurrence. This study supports the integration of predictive analytics with image-based detection systems to enhance decision-making in precision agriculture.

In addition, several researchers have utilized transfer learning techniques with pre-trained models such as VGG, ResNet, and Inception to improve classification accuracy while reducing training time. These models have proven effective in detecting diseases across different plant species by leveraging large-scale image datasets.

From the existing literature, it is evident that while significant progress has been made in plant disease detection using image processing and deep learning, there is still a need for an integrated system that provides real-time detection, user-friendly interfaces, and actionable insights for farmers. Most existing systems focus primarily on classification accuracy without addressing deployment challenges, accessibility, and practical usability in real-world agricultural environments.

1.3 Objectives

The primary objective of the proposed Plant Disease Detection System using Leaf Images is to develop an intelligent and efficient solution for identifying plant diseases at an early stage using image processing and deep learning techniques. The system aims to assist farmers and agricultural practitioners by providing accurate disease detection, thereby improving crop health and productivity.

The specific objectives of the project are as follows:

1. To develop a system that allows users to upload or capture images of plant leaves for disease detection.
2. To preprocess leaf images using techniques such as resizing, noise removal, and segmentation to enhance image quality.
3. To design and implement a Convolutional Neural Network (CNN) model for accurate classification of plant diseases.

4. To train the model using a labeled dataset of healthy and diseased leaf images for multiple plant species.
5. To identify and classify different types of plant diseases based on visual symptoms such as spots, discoloration, and texture variations.
6. To provide real-time prediction results along with the name of the detected disease.
7. To suggest possible remedies, preventive measures, or treatments for the identified plant disease.
8. To develop a user-friendly interface that can be easily used by farmers and agricultural stakeholders.
9. To ensure the system is efficient, scalable, and capable of being deployed on web or mobile platforms.
10. To promote awareness about plant disease management and encourage the adoption of smart agricultural practices.

1.4 Significance

The proposed Plant Disease Detection System using Leaf Images holds significant importance in addressing the challenges faced in modern agriculture, particularly the early identification and management of plant diseases. In many agricultural practices, farmers rely on manual observation and expert consultation to diagnose plant diseases, which can be time-consuming, costly, and sometimes inaccurate. The proposed system offers an automated and efficient solution to overcome these limitations.

The system plays a crucial role in enabling early detection of plant diseases by analyzing leaf images using advanced image processing and deep learning techniques. Early diagnosis helps in preventing the spread of diseases, reducing crop losses, and improving overall agricultural productivity. By providing quick and accurate results, the system supports timely decision-making for effective disease management.

Another important aspect of the system is its accessibility and ease of use. By offering a user-friendly interface, farmers and agricultural stakeholders can easily upload leaf images and obtain disease predictions without requiring specialized technical knowledge. This bridges the gap between advanced technological solutions and practical agricultural applications.

Additionally, the system can provide information about the detected disease along with possible remedies and preventive measures. This feature assists farmers in taking appropriate actions to control and manage plant diseases, thereby reducing dependency on excessive pesticide usage and promoting sustainable farming practices.

From a technical perspective, the project demonstrates the effective integration of computer vision, machine learning, and deep learning techniques. It highlights how modern technologies can be leveraged to solve real-world agricultural problems and improve efficiency in the farming sector.

Overall, the proposed system is significant as it not only enhances disease detection accuracy but also contributes to improved crop yield, reduced economic losses, and the advancement of smart agriculture practices.

1.5 Research Design

The research design for the proposed Plant Disease Detection System using Leaf Images is based on a systematic and structured approach that integrates image processing, machine learning, and deep learning techniques to accurately identify plant diseases. The system is designed to provide an efficient and automated solution for disease detection using visual features extracted from leaf images.

The proposed system follows a client-server architecture, where the frontend provides an interface for users to upload or capture images of plant leaves, and the backend processes the images and generates predictions. The overall design is modular, ensuring that each component of the system performs a specific function effectively.

The research begins with the collection of a dataset consisting of healthy and diseased plant leaf images. These images are preprocessed using techniques such as resizing, normalization, and noise reduction to improve image quality. Image segmentation methods may also be applied to isolate the leaf region from the background, ensuring accurate feature extraction.

Once the images are preprocessed, feature extraction and classification are performed using a Convolutional Neural Network (CNN) model. The model is trained on labeled datasets to learn distinguishing features such as color, texture, and patterns associated with different plant diseases. Transfer learning techniques using pre-trained models may also be employed to enhance performance and reduce training time.

After classification, the system generates the predicted disease label along with confidence scores. The output may also include additional information such as disease description, possible causes, and recommended treatments or preventive measures to assist users in decision-making.

The development process follows a phased approach, including requirement analysis, system design, implementation, training, testing, and evaluation. The application can be developed using modern technologies such as Python for backend processing, along with frameworks like TensorFlow or PyTorch for model development. A web or mobile-based interface can be implemented for user interaction.

This research design ensures that the system is scalable, efficient, and user-friendly, while effectively addressing the problem of plant disease detection and contributing to the advancement of smart agriculture.

Chapter 2 : Requirements Specification

2.1 User Characteristics

The proposed Plant Disease Detection System using Leaf Images is designed for a wide range of users, primarily focusing on individuals involved in agriculture and plant care. The system aims to provide an easy-to-use and accessible platform for users with varying levels of technical expertise, enabling efficient identification of plant diseases.

The main user characteristics are as follows:

1. Farmers and Agricultural Workers

The primary users of the system are farmers who need quick and accurate identification of plant diseases in their crops. These users may not have technical knowledge of plant pathology; therefore, the system provides simple and clear results along with possible remedies to assist in decision-making.

2. Agricultural Experts and Researchers

Experts in agriculture and plant science can use the system as a supportive tool for disease identification and analysis. It can help in validating observations and conducting research related to crop health and disease patterns.

3. Students and Learners

Students studying agriculture, botany, or related fields can use the application as an educational tool to understand different plant diseases, their symptoms, and classification methods based on leaf images.

4. Technical Skill Level

The system is designed for users with basic knowledge of operating smartphones or computers. No advanced technical expertise is required, making it accessible to rural and non-technical users.

5. User Interaction

Users interact with the system through a simple interface where they can upload or capture images of plant leaves. The system processes the image and displays the detected disease along with relevant information.

6. Accessibility

The application is accessible across multiple devices such as smartphones, tablets, and computers. This ensures that users can utilize the system conveniently in fields, laboratories, or other environments.

7. Minimal Authentication Requirement

The system may allow usage without mandatory account creation, enabling quick and easy access. However, optional login features can be included for saving history or tracking previous results.

2.2 Functional Requirements

The functional requirements define the core operations and functionalities of the Plant Disease Detection System using Leaf Images. These requirements describe what the system should do to meet user needs and achieve the objectives of accurate and efficient disease identification.

The main functional requirements of the system are as follows:

- 1. FR-1: Image Upload and Capture**

The system shall allow users to upload an image of a plant leaf or capture it using a device camera for disease detection.

- 2. FR-2: Image Preprocessing**

The system shall preprocess the uploaded leaf image using techniques such as resizing, normalization, and noise reduction to improve image quality.

- 3. FR-3: Leaf Segmentation**

The system shall isolate the leaf region from the background using image segmentation techniques to ensure accurate analysis.

- 4. FR-4: Disease Classification**

The system shall classify the leaf image using a trained Convolutional Neural Network (CNN) model to detect the presence and type of plant disease.

- 5. FR-5: Disease Identification**

The system shall identify the specific plant disease based on the classification results and learned patterns.

- 6. FR-6: Result Generation**

The system shall generate prediction results along with confidence scores indicating the accuracy of the detected disease.

- 7. FR-7: Disease Information Display**

The system shall provide detailed information about the detected disease, including symptoms, causes, and effects on the plant.

- 8. FR-8: Remedy and Prevention Suggestions**

The system shall suggest appropriate remedies, treatments, and preventive measures for the identified disease.

9. FR-9: Result Display

The system shall display the uploaded image, detected disease name, confidence score, and recommendations in a clear and user-friendly interface.

10. FR-10: Error Handling

The system shall handle invalid, unclear, or unsupported images and provide appropriate error messages to guide the user.

11. FR-11: Responsive Interface

The system shall provide a responsive interface that works efficiently across various devices such as smartphones, tablets, and desktops.

2.3 Dependencies

The Plant Disease Detection System using Leaf Images depends on several external components, technologies, and system conditions for its proper functioning. These dependencies are essential to ensure accurate image processing, disease classification, and result generation.

The key dependencies of the system are as follows:

1. Deep Learning Frameworks

The system relies on deep learning frameworks such as TensorFlow or PyTorch for building and training the Convolutional Neural Network (CNN) model. The performance of disease detection depends on the efficiency of these frameworks.

2. Pre-trained Models / Transfer Learning

The system may utilize pre-trained models such as VGG, ResNet, or Inception for transfer learning. These models improve accuracy and reduce training time by leveraging previously learned features.

3. Dataset Availability

The system depends on a well-labeled dataset of plant leaf images, including both healthy and diseased samples. The accuracy of the model is highly dependent on the quality, size, and diversity of the dataset.

4. Internet Connectivity

If the system is deployed as a web or cloud-based application, a stable internet connection is required for uploading images, processing data, and retrieving results.

5. **Hardware Resources**

The system may require adequate computational resources such as GPUs or high-performance CPUs for model training and efficient inference. Limited hardware resources may affect performance and response time.

6. **Image Quality**

The accuracy of disease detection depends on the clarity and quality of the input leaf images. Blurred, noisy, or poorly lit images may reduce the effectiveness of the system.

7. **Backend Server Availability**

The system requires a reliable backend server to handle image processing, model inference, and data management. Technologies such as Python-based servers or web frameworks may be used for this purpose.

8. **Web/Mobile Platform Compatibility**

The system depends on compatibility with modern web browsers or mobile platforms to ensure smooth user interaction and accessibility.

2.4 Performance Requirements

The performance requirements define the expected efficiency, responsiveness, and overall behavior of the Plant Disease Detection System using Leaf Images under normal operating conditions. The system is designed to deliver fast, accurate, and reliable results to ensure a smooth and effective user experience.

The key performance requirements are as follows:

1. **Response Time**

The system should process uploaded leaf images and generate disease detection results within a few seconds under normal operating conditions.

2. **Image Processing Efficiency**

The preprocessing and segmentation of leaf images should be performed efficiently with minimal delay, ensuring that the input image is quickly prepared for classification.

3. **Model Inference Speed**

The trained Convolutional Neural Network (CNN) model should classify images rapidly, enabling near real-time prediction of plant diseases.

4. **Concurrent Users Handling**

The system should be capable of handling multiple user requests simultaneously without significant degradation in performance.

5. Scalability

The application should be scalable to support an increasing number of users and large volumes of image data without affecting system performance.

6. System Availability

The system should be available for use at all times with minimal downtime, ensuring continuous accessibility for users.

7. Data Processing Speed

The disease detection, result generation, and recommendation processes should be executed efficiently to provide real-time or near real-time feedback.

8. Error Handling Performance

The system should quickly detect and handle invalid or poor-quality images and provide appropriate feedback to users without delay.

9. Resource Utilization

The system should efficiently utilize computational resources such as CPU, GPU, and memory to maintain optimal performance across different devices and platforms.

2.5 Hardware Requirements

The Plant Disease Detection System using Leaf Images does not require highly specialized or expensive hardware for its operation. The system is designed to be accessible on commonly available devices, ensuring ease of use for farmers, students, and agricultural practitioners.

The hardware requirements are as follows:

1. User Device

The application can be accessed on standard devices such as smartphones, tablets, laptops, or desktop computers. These devices should have basic processing capability and internet connectivity.

2. Camera Support

A device with a camera is recommended for capturing images of plant leaves directly from the field. Alternatively, users can upload images stored on their device.

3. Display Unit

A standard display screen is required to view the user interface, uploaded images, and disease detection results clearly.

4. Input Devices

Basic input devices such as a touchscreen, keyboard, or mouse are required for interacting with the system.

5. Server Hardware

The backend system requires a server with sufficient processing power, memory, and storage to handle image processing, model inference, and multiple user requests efficiently. For training deep learning models, systems equipped with GPUs are recommended to improve performance.

2.6 Software Requirements

The Plant Disease Detection System using Leaf Images is developed using modern software technologies and requires specific tools and frameworks for its development, training, deployment, and execution. These software components ensure efficient processing, accurate disease detection, and smooth user interaction across different platforms.

The key software requirements are as follows:

1. Operating System

The application can run on any modern operating system such as Windows, macOS, or Linux, as it can be deployed as a web or mobile-based application.

2. Frontend Technologies

The user interface is developed using web technologies such as HTML, CSS, and JavaScript. Frameworks like React may be used to enhance interactivity, responsiveness, and overall user experience.

3. Backend Technologies

The server-side logic is implemented using Python (Flask/Django) or Node.js, which handles image processing, model inference, API requests, and communication between system components.

4. Deep Learning Frameworks

Frameworks such as TensorFlow or PyTorch are used to design, train, and deploy the Convolutional Neural Network (CNN) model for plant disease classification.

5. Image Processing Libraries

Libraries such as OpenCV, NumPy, and PIL (Python Imaging Library) are used for preprocessing tasks like image resizing, normalization, and segmentation.

6. Dataset and Model Training Tools

Tools such as Jupyter Notebook or Google Colab may be used for training and testing the deep learning model using labeled datasets of plant leaf images.

7. Database Management System

A database system such as MongoDB or MySQL may be used to store disease information, prediction results, and user-related data if required.

8. Web Browser

The application requires a modern web browser such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari for proper functionality and user interaction.

9. Development Tools

Tools such as Visual Studio Code (VS Code), Git, and package managers like pip or npm are used for development, version control, and dependency management.

10. API Integration (Optional)

The system may integrate external APIs for additional agricultural information, weather data, or disease management resources to enhance system functionality.

Chapter 3: Design

3.1 Algorithm

The following algorithm describes the working process of the Plant Disease Detection System using Leaf Images:

BEGIN

Step 1: Initialize the system

Step 2: User uploads or captures an image of a plant leaf

Step 3: Validate input image

IF the image is invalid (wrong format, size, or corrupted)
THEN display error message and **STOP**
ENDIF

Step 4: Send the image to the backend server

Step 5: Perform backend validation

IF server error or request failure occurs
THEN return error response and **STOP**
ENDIF

Step 6: Store image temporarily (local/cloud storage)

Obtain image path or URL

Step 7: Perform image preprocessing

- Resize image
- Normalize pixel values
- Remove noise

Step 8: Perform leaf segmentation

Isolate leaf region from background
IF leaf not detected
THEN return error message and **STOP**
ENDIF

Step 9: Load trained CNN model

Step 10: Perform disease classification

- Extract features from image
- Predict disease class

Step 11: Generate prediction results

- Disease name
- Confidence score

Step 12: Retrieve disease information from database

- Symptoms
- Causes
- Remedies and prevention

Step 13: Store prediction results (optional)

Save image, prediction, and metadata

Step 14: Delete temporary image (if stored in cloud)

Step 15: Send response to frontend

Step 16: Display results to user

- Detected disease
- Confidence score
- Disease details
- Suggested remedies

Step 17: (Optional) Allow user to analyze another image

END

3.2 Function Oriented Design for Procedural Approach

The Plant Disease Detection System using Leaf Images follows a function-oriented design approach, where the system is divided into multiple functional modules. Each module is responsible for performing a specific task, and together they form a complete workflow for detecting plant diseases efficiently.

The system is designed in a procedural manner, where the output of one module serves as the input to the next, ensuring a smooth and logical flow of data from user input to final result generation.

The major functional modules of the system are described below:

1. Image Input Module

This module allows users to upload or capture an image of a plant leaf through the frontend interface. It performs basic validation such as checking file type, size, and format before sending the image to the backend for processing.

2. Image Upload and Storage Module

The backend receives the image and temporarily stores it in local or cloud storage. This module ensures secure handling and efficient access to the image during processing.

3. Image Preprocessing Module

This module prepares the image for analysis by applying preprocessing techniques such as resizing, normalization, and noise removal to enhance image quality.

4. Leaf Segmentation Module

It This module isolates the leaf region from the background using segmentation techniques. ensures that only the relevant portion of the image is used for disease detection.

5. Feature Extraction and Classification Module

This is the core module of the system. It uses a trained Convolutional Neural Network (CNN) to extract features such as color, texture, and patterns from the leaf image and classify it into healthy or diseased categories.

6. Disease Identification Module

Based on the classification results, this module identifies the specific type of plant disease affecting the leaf.

7. Analysis Module

This module retrieves relevant information about the detected disease, including symptoms, causes, effects, and recommended treatments or preventive measures.

8. Database Module

This module stores and retrieves data such as disease details, model outputs, and optionally user queries. It ensures efficient data management and quick access to information.

9. Response Module

The backend compiles the processed results and sends a structured response (e.g., JSON format) to the frontend.

10. Presentation Module

The frontend displays the results in a user-friendly format, including:

- Detected disease name
- Confidence score
- Disease description

- Causes and symptoms
- Remedies and preventive measures

3.3 System Design

The Plant Disease Detection System using Leaf Images is designed using a client–server architecture, where the system is divided into frontend and backend components that communicate through HTTP requests. This design ensures modularity, scalability, and efficient data processing.

The frontend is developed using modern web or mobile technologies and is responsible for handling user interactions such as image upload, input validation, and displaying detection results. It provides a simple, responsive, and user-friendly interface that allows users to easily capture or upload plant leaf images and view disease analysis results.

The backend acts as the core processing unit of the system. It handles image upload, validation, preprocessing, and communication with the trained deep learning model. The backend processes the input image using image processing techniques and passes it to a Convolutional Neural Network (CNN) model for disease classification.

The system follows a structured workflow where the uploaded image is first preprocessed to improve quality through resizing, normalization, and noise removal. The leaf region is then segmented from the background to ensure accurate feature extraction. The processed image is fed into the trained CNN model, which predicts the disease class based on learned features such as color, texture, and patterns.

Once the disease is identified, the system retrieves relevant information from the database, including disease description, causes, symptoms, and recommended remedies. The results are then formatted into a structured response and sent back to the frontend for display.

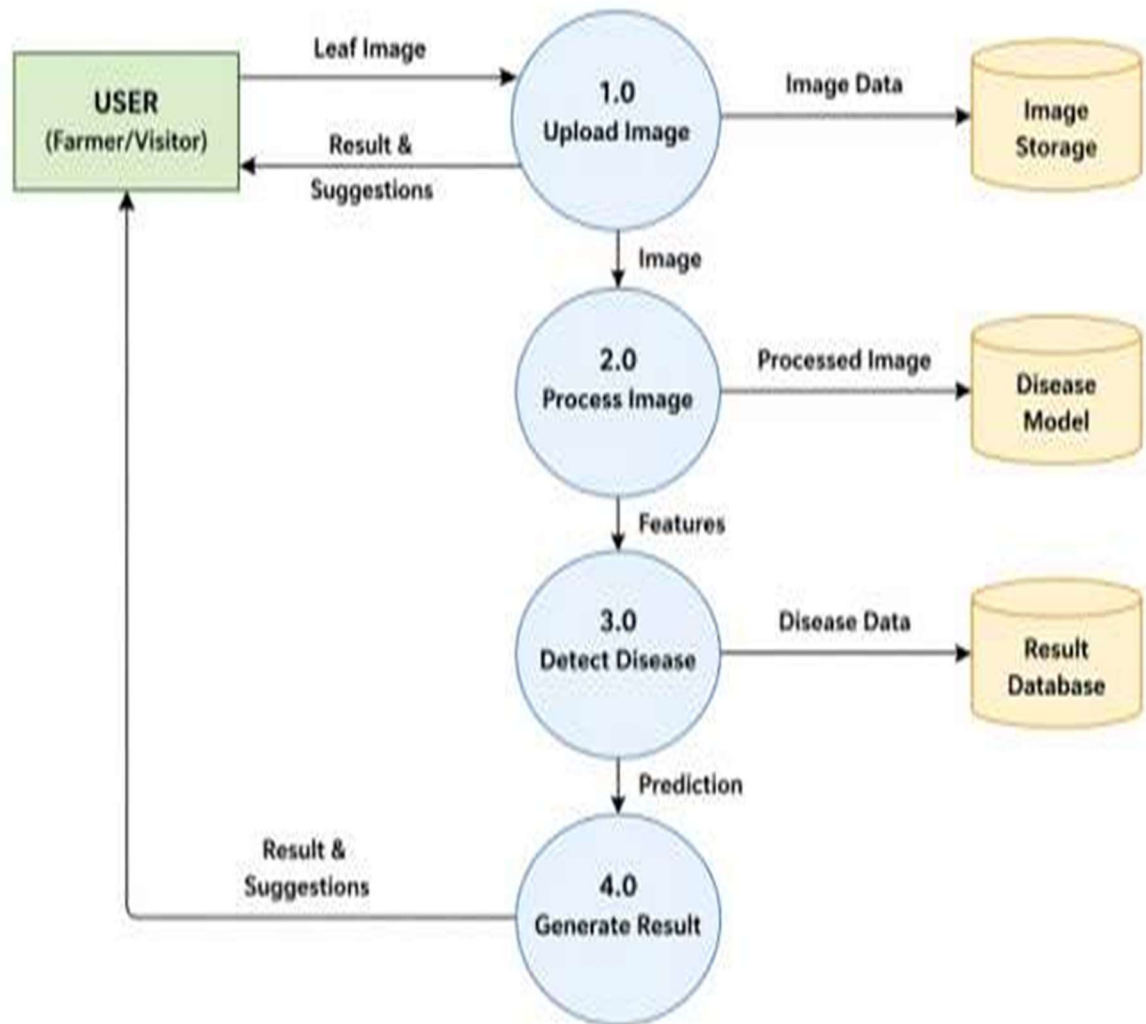
The database design is simple and efficient, storing information related to plant diseases, their characteristics, and treatment methods. This ensures quick retrieval of relevant information without complex relationships, making the system lightweight and easy to maintain.

The final processed data is presented to the user in an organized format, including the detected disease name, confidence score, and suggested remedies. The system may also support additional features such as repeated analysis for multiple images.

Overall, the system design ensures efficient data flow, accurate disease detection, and seamless integration of image processing and deep learning techniques, making the application scalable, reliable, and user-friendly.

3.3.1 DATA FLOW DIAGRAM (DFD)

Data Flow Diagram (DFD) – LeafScan Plant Disease Detection System

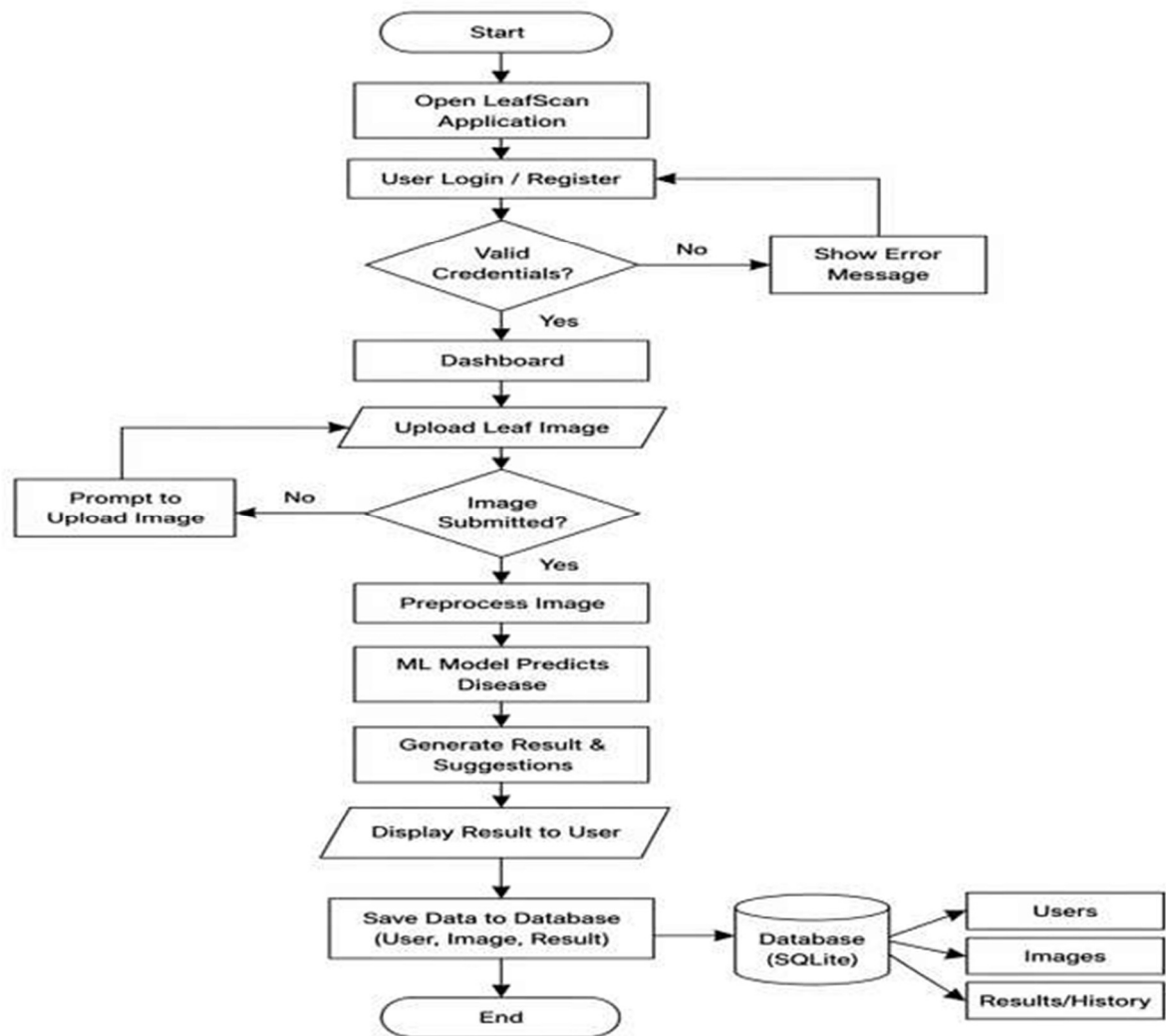


- Upload Image: User uploads leaf image
- Process Image: Image is cleaned and prepared
- Detect Disease: Model predicts disease
- Generate Result: Shows result + suggestions

3.3.2 ACTIVITY DIAGRAM



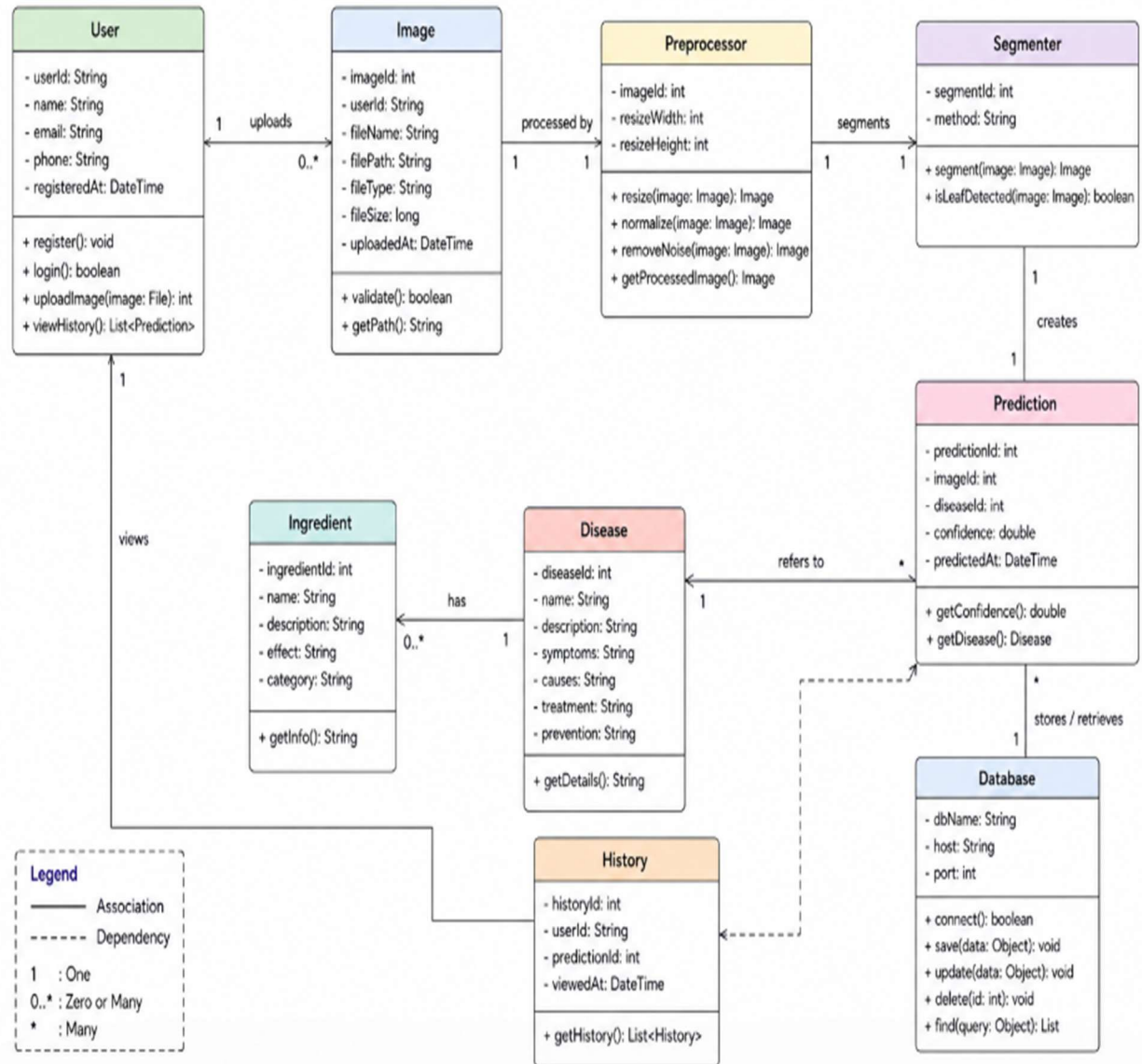
3.3.3 FLOWCHART



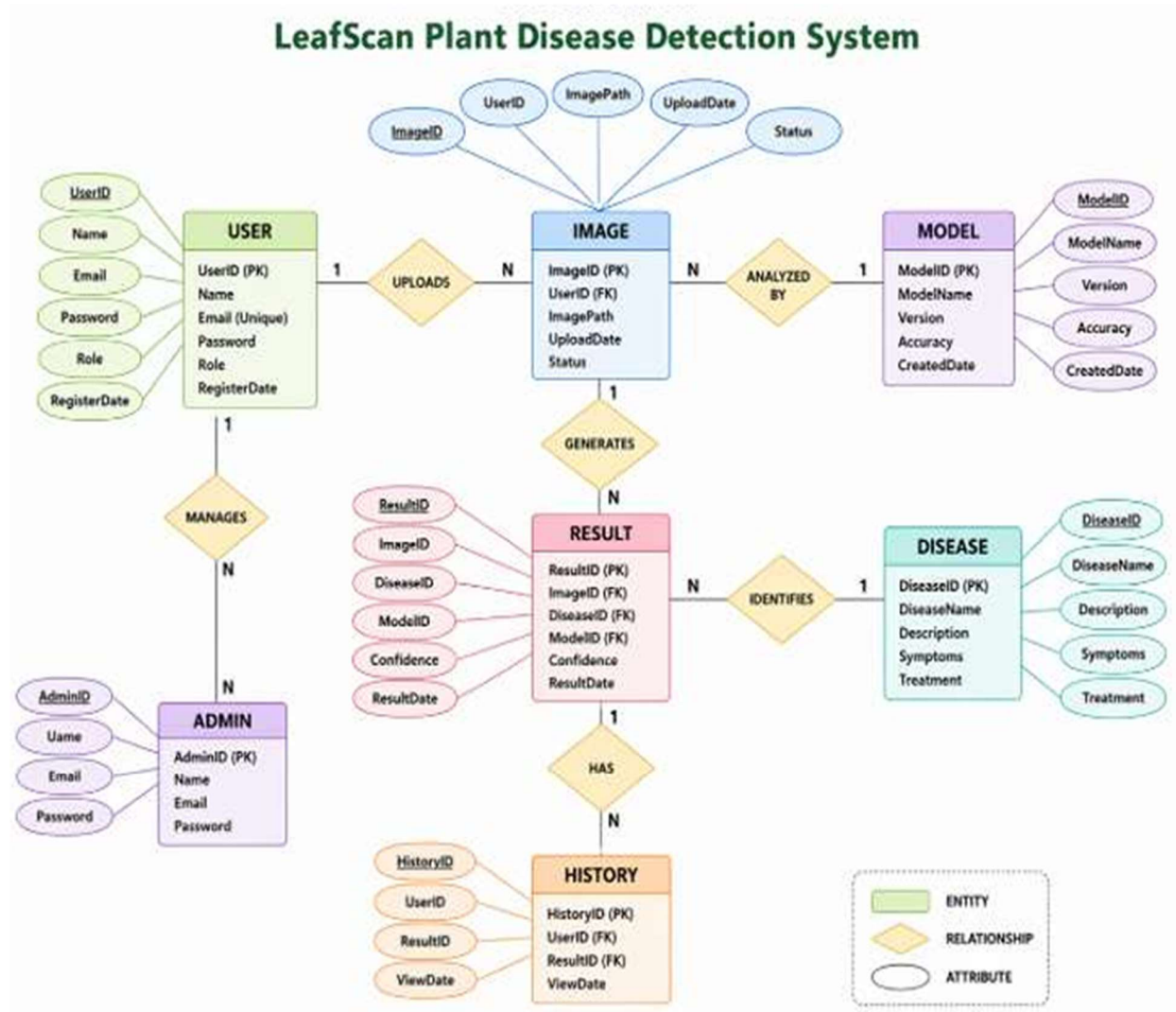
- Start → Open application
- Login/Register → Verify user
- Upload Image → Submit leaf image
- Process Image → Prepare data
- Detect Disease → ML prediction
- Generate Result → Show suggestions
- Save Data → Store in database
- End

3.3.4 CLASS DIAGRAM

Class Diagram – Plant Disease Detection using Leaf Images

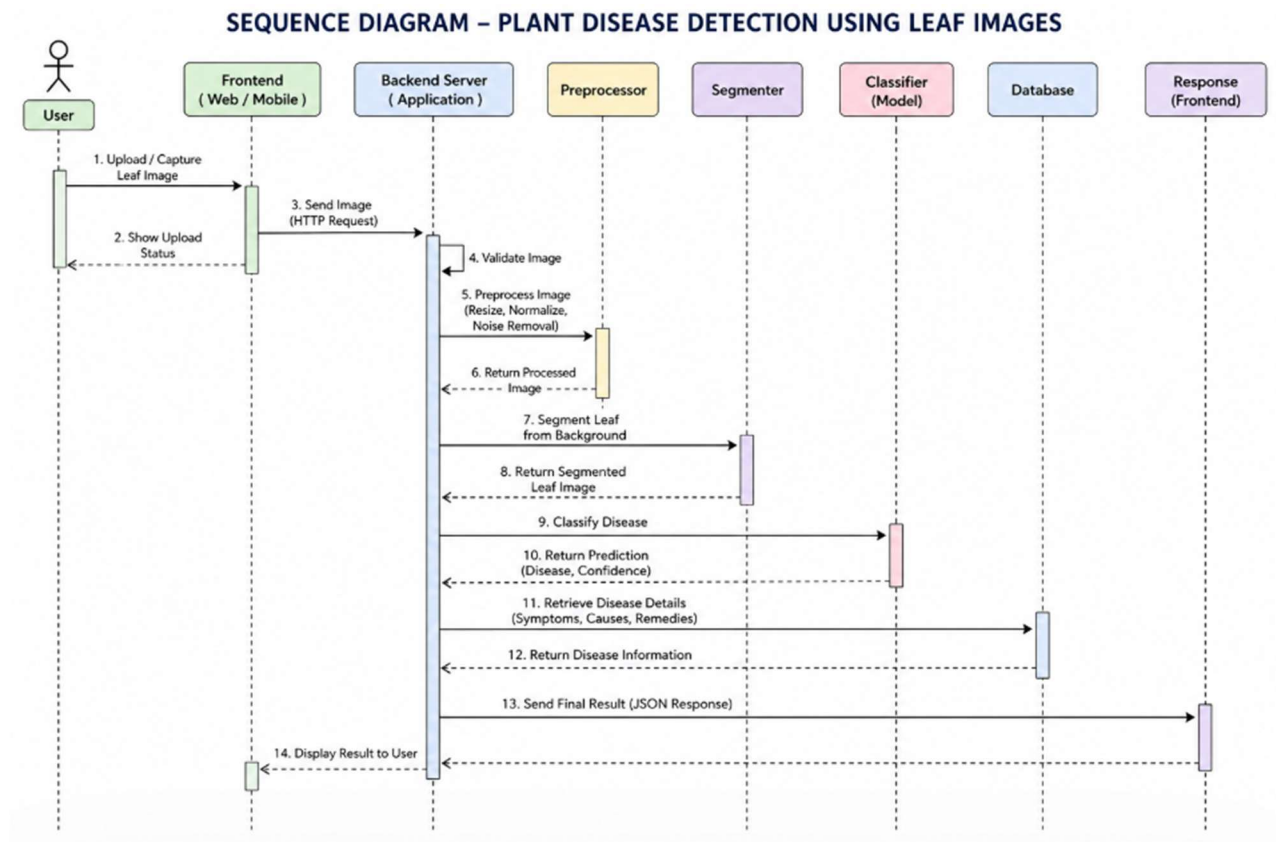


3.3.5 ER DIAGRAM



- User → Uploads → Image
- Image → Analyzed by → Model
- Image → Generates → Result
- Result → Identifies → Disease
- Result → Stored in → History
- Admin → Manages → Users & System

3.3.6 SEQUENCE DIAGRAM



3.4 Database Design

The database design of the Plant Disease Detection System using Leaf Images is structured to efficiently store, retrieve, and manage plant disease-related data and prediction results. The system may use a NoSQL database such as MongoDB, which provides flexibility in handling semi-structured data such as image metadata, prediction outputs, and disease information.

The database is designed to support fast data retrieval and efficient storage. It may also include a caching mechanism to reduce redundant processing and improve overall system performance when similar inputs are encountered.

3.4.1 Logical Database Design

The logical database design represents the conceptual structure of the database, including entities and their attributes used in the Plant Disease Detection System.

1. Disease Collection

This collection stores detailed information about different plant diseases.

Attributes:

- `_id` (Primary Key)
- `diseaseName` (Name of the disease)
- `plantType` (Type of plant affected)
- `symptoms` (List of symptoms)
- `causes` (Reasons for the disease)
- `treatment` (Suggested remedies)
- `prevention` (Preventive measures)
- `createdAt` (Timestamp)
- `updatedAt` (Timestamp)

2. Prediction Collection

This collection stores the results generated after analyzing leaf images.

Attributes:

- `_id` (Primary Key)
- `imagePath` (Path or URL of uploaded image)
- `predictedDisease` (Detected disease name)
- `confidenceScore` (Prediction accuracy value)
- `plantType` (Detected or selected plant type)
- `createdAt` (Timestamp)
- `updatedAt` (Timestamp)

3. Image Metadata Collection (Optional)

This collection stores additional details about uploaded images.

Attributes:

- `_id` (Primary Key)
- `imageName` (Name of the image file)

- imageSize (Size of image)
- imageFormat (File format, e.g., JPG, PNG)
- uploadTime (Timestamp)
- processingStatus (Processed / Pending / Failed)

Relationship

There is no strict foreign key relationship between the collections in a NoSQL database. However, logically:

- One disease can be associated with multiple predictions
- Each prediction corresponds to one detected disease
- Image metadata is linked to prediction records through image references

Chapter 4: Implementation, Testing and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies used for Implementation

The development of the Plant Disease Detection System using Leaf Images involves the use of modern programming languages, development tools, and technologies. These technologies are carefully selected to ensure efficient image processing, accurate disease prediction, scalability, and overall system performance.

1. Programming Languages

- **Python**
Used as the primary programming language for implementing the machine learning and image processing components of the system. It provides strong support for deep learning frameworks and scientific computing.
- **JavaScript**
Used for developing the frontend and handling dynamic user interactions in the web application.
- **HTML**
Used to structure the content of the web interface.
- **CSS**
Used to design and style the user interface, ensuring responsiveness and visual appeal.

2. Frontend Technologies

- **React.js**
A JavaScript library used to build a dynamic and responsive user interface.
- **Vite / Create React App**
Used as a development environment and build tool for faster frontend development.
- **Bootstrap / Material UI**
Provides pre-designed components to create a modern and user-friendly interface.

3. Backend Technologies

- **Python (Flask / Django)**
Used to build the backend server that handles image processing, model inference, and API communication.

- Node.js
Can be used for handling API requests and managing server-side operations.

4. Machine Learning and Image Processing

- TensorFlow / Keras
Used to design, train, and deploy the Convolutional Neural Network (CNN) model for plant disease classification.
- OpenCV
Used for image preprocessing tasks such as resizing, filtering, and segmentation.
- NumPy & Pandas
Used for numerical computations and data handling.

5. Database

- MongoDB & MySQL
Used to store plant disease information, prediction results, and related data.

6. APIs and External Services

- Cloud & Storage
Used for temporary storage of uploaded leaf images.
- Agricultural APIs
Used to fetch additional information such as disease remedies, weather data, or crop-related insights.

7. Development Tools

- Visual Studio Code
Used as the primary code editor for development.
- Git & GitHub
Used for version control and project management.
- Jupyter Notebook / Google Colab
Used for training and testing the machine learning model.

8. Security and Middleware

- CORS
Enables secure communication between frontend and backend.
- Authentication Middleware (Optional)
Used to manage user access and security if required.
- File Handling Libraries
Used to manage image uploads and processing in the backend.

This combination of technologies ensures that the system is efficient, scalable, accurate, and easy to use, making it suitable for real-world plant disease detection applications.

4.2 Testing Techniques and Test Plans

Testing is a crucial phase in the development of the Plant Disease Detection System using Leaf Images. It ensures that the system functions correctly, efficiently, and meets user requirements. Various testing techniques were applied to verify the performance and reliability of different components of the system.

1. Testing Techniques Used

a) Unit Testing

Each individual module such as image upload, preprocessing, leaf segmentation, and disease prediction was tested independently to ensure correct functionality.

b) Integration Testing

Integration testing was performed to verify the interaction between different modules, such as frontend and backend communication, backend and machine learning model integration, and database operations.

c) System Testing

The complete system was tested as a whole to ensure that all components work together seamlessly, from image upload to disease detection and result display.

d) User Interface Testing

The frontend interface was tested to ensure proper layout, responsiveness, and user-friendly interaction across different devices such as mobiles, tablets, and desktops.

e) Performance Testing

The system was tested for response time and efficiency, especially during image processing and model inference, to ensure smooth performance under normal conditions.

f) Accuracy Testing

The prediction accuracy of the model was evaluated using test datasets to ensure reliable disease detection results.

g) Error Handling Testing

Different error scenarios such as invalid image input, poor image quality, and processing failures were tested to ensure appropriate error messages are displayed to the user.

2. Test Plan

The test plan defines different test cases to validate system functionality. Key test scenarios include:

- Uploading a valid leaf image and verifying correct disease prediction
- Uploading an invalid or corrupted image and checking error handling
- Testing system response time for image processing
- Verifying correctness of displayed disease information and remedies
- Testing system behavior under multiple user requests
- Checking compatibility across different devices and browsers

Table 4.1: Test Cases for LeafScan Plant Disease Detection System

Test Case Table

Test Case ID	Test Description	Input	Expected Output	Result
TC-01	Upload valid image	Close-up leaf image with leaf spot	Image uploaded and displayed successfully	Pass
TC-02	Upload invalid file type	Text file (leaf_data.txt)	Error message: "Invalid file format, please upload an image."	Pass
TC-03	Image with no leaf	Photo of a stone wall	System returns: "No leaf detected. Please try again."	Pass
TC-04	Healthy leaf detection	Clear image of a healthy green leaf	Result: "Healthy - No disease detected."	Pass
TC-05	Disease Identification (Powdery Mildew)	Leaf image with white powdery residue	Correct classification as "Powdery Mildew".	Pass
TC-06	Disease Identification (Black Spot)	Leaf image with small, dark spots	Correct classification as "Black Spot".	Pass
TC-07	Low-light condition	Blurred, dark leaf image	Pre-processing enhances and still attempts analysis; results may be less confident.	Pass
TC-08	High-light/glare condition	Overexposed leaf image	System attempts analysis but may flag for re-take due to glare.	Pass
TC-09	Multiple diseased leaves in one image	Photo of a plant with multiple affected leaves	Model isolates individual leaves or provides summary; ideally flags. (Simpler here): System flags multiple issues.	Pass
TC-10	Recommendation Display	Post-classification result	Displays recommended treatment options.	Pass

4.3 Installation Instructions

The following steps describe how to install and run the Plant Disease Detection System using Leaf Images on a local system.

1. Prerequisites

Before installation, ensure that the following software and tools are installed on the system:

- Python (version 3.8 or above)
- pip (Python Package Manager)
- Node.js (optional, if frontend is used separately)
- Git (optional, for cloning repository)
- Code Editor (Visual Studio Code recommended)
- Jupyter Notebook / Google Colab (for model training, optional)

2. Clone the Repository

```
git clone <your-repository-link>
```

```
cd plant-disease-detection
```

3. Install Dependencies

Backend Setup (Python):

```
cd backend
```

```
pip install -r requirements.txt
```

Frontend Setup (if applicable):

```
cd frontend
```

```
npm install
```

4. Environment Configuration

Create a .env file in the backend directory and add the following:

```
PORT=5000
```

```
MODEL_PATH=path_to_trained_model
```

```
DATABASE_URI=your_database_connection_string
```

```
SECRET_KEY=your_secret_key
```

5. Run the Application

Start Backend Server:

```
cd backend
```

```
python app.py
```

Start Frontend (if applicable):

```
cd frontend
```

```
npm start
```

6. Access the Application

Open your browser and go to:

```
http://localhost:3000
```

(or the port specified in your project)

7. Optional Deployment

The application can be deployed using:

- **Frontend:** Vercel / Netlify
- **Backend:** Render / Railway / Heroku
- **Model Hosting (optional):** Cloud platforms such as AWS or Google Cloud

4.4 End User Instructions

The Smart Food Ingredient Analyzer and Nutrition Awareness Web Application is designed to be simple, intuitive, and user-friendly. The following steps guide users on how to effectively use the system.

1. Access the Application

- Open any modern web browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge).
- Enter the application URL in the address bar and navigate to the homepage.

2. Upload Food Label Image

- Click on the “**Upload Image**” button.
- Select a food product label image from your device or drag and drop it into the upload area.
- Ensure that the image is clear, well-lit, and readable for accurate analysis.

3. Analyze the Product

- Click on the “**Analyze**” button.
- The system will process the uploaded image using OCR and extract ingredient information.

4. View Results

After processing, the system displays:

- Health Score of the product
- Simplified Ingredient Explanations
- Possible Health Effects
- Daily Consumption Verdict
- Alternative Product Suggestions

5. Compare Products (Optional Feature)

- Navigate to the **Compare Section**.
- Upload two different product images.

- View a side-by-side comparison along with a recommended healthier option.

6. Error Handling

- If the uploaded image is unclear or invalid, the system will display an error message.
- Users should upload a clearer image for better and more accurate results.

7. Tips for Best Results

- Use high-quality, well-focused images.
- Ensure ingredient text is clearly visible.
- Avoid blurred, cropped, or low-resolution images.

Chapter 5: Results and Discussions

5.1 User Interface Representation

The user interface of the Smart Food Ingredient Analyzer and Nutrition Awareness Web Application is designed to be simple, intuitive, and visually appealing. It ensures that users can easily upload food label images, analyze ingredients, and understand results without requiring technical expertise.

1. Home / Analyzer Interface

The main interface provides a clean and modern layout where users can upload food label images. It includes a drag-and-drop upload area, making the process user-friendly. The design focuses on simplicity to ensure that even non-technical users can easily interact with the system.

2. Image Preview and Controls

After uploading an image, the system displays a preview along with options to edit or remove the image. This allows users to verify the selected image before proceeding with analysis.

3. Processing / Loading State (with Quiz Feature)

During processing, the system displays a loading interface with a progress indicator. Additionally, an interactive quiz is presented to engage users while the system processes the image. This improves user experience by reducing perceived waiting time.

4. Analysis Result Interface

Once analysis is complete, the system displays detailed results including:

- Health Score
- Simplified Ingredients
- Possible Effects
- Daily Use Verdict
- Better Alternatives
- Detected Ingredients

The results are presented in a structured, card-based layout for better readability and user experience.

5. Key UI Features

- Clean and modern design
- Drag-and-drop image upload
- Real-time feedback during processing
- Interactive quiz during loading
- Structured and readable result display
- Responsive design for multiple devices

6. Product Comparison Interface

The application provides a dedicated comparison interface where users can upload two different food product labels and analyze them simultaneously.

Each product panel supports independent image upload, preview, and analysis. After processing both products, the system generates a comparative result highlighting the healthier option based on multiple health indicators.

7. Comparison Result Display

The comparison results are presented in a structured format including:

- Winner Declaration (e.g., Product A or Product B)
- Health Score Comparison
- Additive Risk Analysis
- Sugar Risk Analysis
- Detailed Explanation for Decision

The results are displayed in a tabular or side-by-side format for easy understanding.

8. Additional Features in UI

- Independent analysis for each product
- Real-time comparison after both analyses
- Cache indication (e.g., “Fetched from cache”)
- Quiz score tracking for engagement
- Clean side-by-side layout for better usability

5.2 Brief Description of Various Modules of the System

The Smart Food Ingredient Analyzer and Nutrition Awareness Web Application is divided into multiple functional modules. Each module is responsible for a specific task, ensuring modularity, scalability, and efficient system performance.

1. User Interface Module (Frontend)

This module handles all user interactions.

Responsibilities:

- Image upload (drag-and-drop or file selection)
- Display of analysis results
- Display of comparison results
- Showing loading state and quiz interaction
- Error handling and user feedback

2. Image Upload & Validation Module

This module ensures that only valid images are processed.

Responsibilities:

- Accept image input (JPG, JPEG, PNG)
- Validate file size and format
- Provide image preview
- Handle invalid upload errors

3. OCR Processing Module

This module extracts text from the uploaded food label image.

Responsibilities:

- Send image to Google Vision API
- Extract raw text from the image
- Return detected text for further processing

4. Ingredient Extraction & Cleaning Module

This module processes raw OCR text to identify meaningful ingredients.

Responsibilities:

- Identify ingredient section from OCR text
- Clean and normalize extracted text
- Remove noise and duplicates
- Generate structured ingredient list

5. Caching Module (MongoDB)

This module improves performance by avoiding repeated analysis.

Responsibilities:

- Generate hash from ingredient list
- Check if analysis already exists
- Return cached result if available
- Reduce API calls and processing time

6. AI Analysis Module (LLM-Based)

This module performs intelligent analysis of ingredients.

Responsibilities:

- Generate simplified ingredient explanations
- Calculate health score
- Identify possible health effects
- Provide daily consumption verdict
- Suggest better alternatives

7. Database Management Module

This module handles storage and retrieval of data.

Responsibilities:

- Store analyzed product data
- Maintain ingredient information
- Manage cache records

- Perform database queries

8. Comparison Module

This module enables comparison between two food products.

Responsibilities:

- Perform independent analysis on two products
- Compare health scores and risks
- Identify better product
- Generate comparison explanation

9. API & Backend Control Module

This module acts as the central controller of the system.

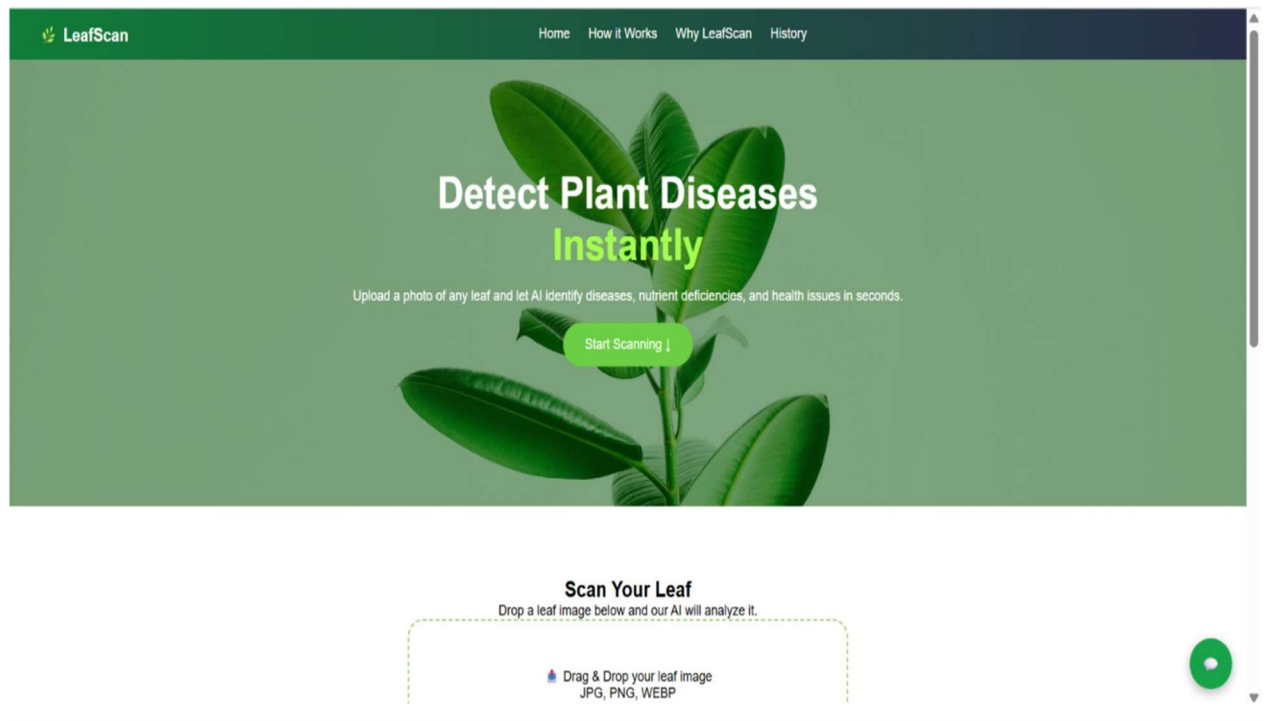
Responsibilities:

- Handle API requests (e.g., /api/upload)
- Coordinate between modules
- Manage communication with external services

5.3 Snapshots of System with Brief Detail of Each

This section presents the working interface of the **LeafScan – Plant Disease Detection Web Application**. Each snapshot demonstrates a specific functionality of the system along with its description.

Figure 5.1: Home / Landing Page



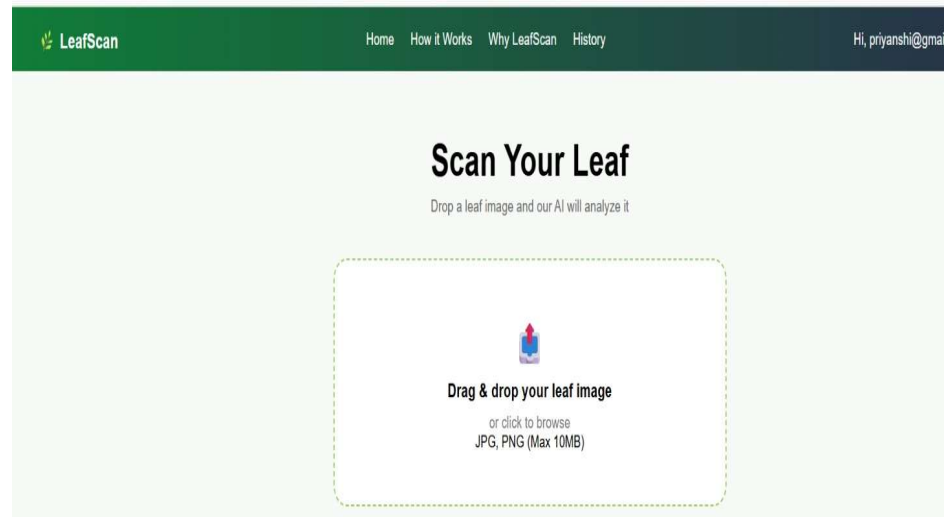
Description:

This is the main landing page of the application.

- Displays the application name *LeafScan*
- Highlights the main feature: *Detect Plant Diseases Instantly*
- Includes navigation bar (Home, How it Works, Why LeafScan, History)
- Contains a “**Start Scanning**” button to begin the process
- Provides an overview of the system’s purpose

This screen acts as the entry point for users.

Figure 5.2: Image Upload Interface



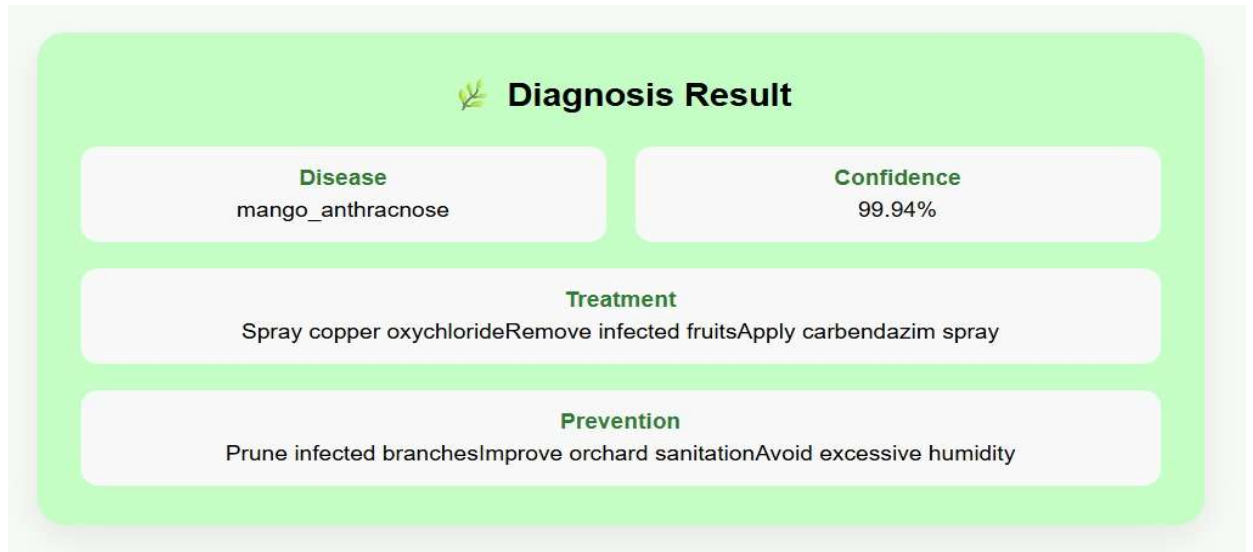
Description:

This screen allows users to upload an image of a leaf for analysis.

- Drag-and-drop upload functionality
- Supports formats like JPG, PNG, WEBP
- Displays file size limit (e.g., Max 10MB)
- Includes an **“Analyze Plant”** button

This is where the user provides input to the system.

Figure 5.3: Analysis Result Screen



Description:

After processing, the system displays the diagnosis results.

- Detected disease name (e.g., *mango powdery mildew*)
- Confidence score (e.g., 99.79%)
- Suggested treatment methods
- Prevention tips for the disease

This is the core output of the system where users get actionable insights.

Figure 5.4: How It Works Section



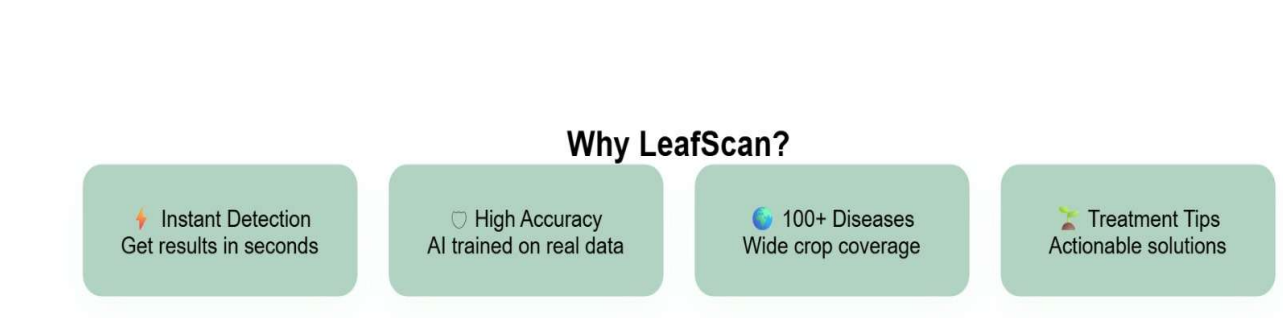
Description:

This section explains the working process of the system in a simple 3-step flow:

1. Capture – Upload leaf image
2. Analyze – AI detects disease
3. Results – Get diagnosis and treatment

Helps users understand the workflow quickly.

Figure 5.5: Why LeafScan Section



Description:

This section highlights key features and advantages of the system:

- Instant detection (results in seconds)
- High accuracy (AI trained on real data)
- Wide disease coverage
- Actionable treatment tips

Builds user trust and explains system benefits.

5.4 Back Ends Representation

```
backend > app.py > ...
You, 10 minutes ago | 1 author (You)
1 from flask import Flask, request, render_template, jsonify
2 from flask_cors import CORS
3 import tensorflow as tf
4 import numpy as np
5 from PIL import Image
6 import json
7 from tensorflow.keras.applications.efficientnet import preprocess_input
8 from openai import OpenAI
9 import os
10 import sqlite3
11 from datetime import datetime
12 from dotenv import load_dotenv
13 from google import genai
14
15
16 from flask import send_from_directory
17
18
19 client = genai.Client(api_key="AIzaSyDBAB1_7kw9X_PDXBJXRpc8NP-gUZwRkyM")
20
21 # Load env
22 load_dotenv()
23 #AIzaSyDBAB1_7kw9X_PDXBJXRpc8NP-gUZwRkyM
24 app = Flask(__name__)
25 CORS(app, resources={r"/**": {"origins": "*"}})
26 You, yesterday * Initial clean commit
27
28 # OpenAI
29 client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
30
31 # Load model
32 disease_model = tf.keras.models.load_model("plant_disease_model.h5", compile=False)
33
34 # Load class mapping
35 with open("classes.json", "r") as f:
36     class_indices = json.load(f)
37
38 classes = {v: k for k, v in class_indices.items()}
39
40 # Load disease info
41 with open("disease_info.json") as f:
42     disease_info = json.load(f)
43
44
```

Figure 5.6: Back End Representation

5.5 Snapshots of Database Tables with brief description

database.db									
Rows: 103									
	id	user_email	disease	confidence	#	image		date	
1	1	user@example.com	potato_nematode	0.8716305494308472	IMG_8339.JPG			08/03/2026 18:00:47	
2	2	user@example.com	mango_powdery_mildew	0.9972237348556519	20211109_121946 (Custom).jpg			08/03/2026 18:10:45	
3	3	user@example.com	mango_powdery_mildew	0.9972237348556519	20211109_121946 (Custom).jpg			08/03/2026 18:10:45	
4	4	user@example.com	mango_anthracnose	0.9856699109077454	20211011_133430 (Custom).jpg			10/03/2026 11:38:33	
5	5	user@example.com	mango_anthracnose	0.9856699109077454	20211011_133430 (Custom).jpg			10/03/2026 11:38:33	
6	6	user@example.com	mango_anthracnose	0.9856699109077454	20211011_133430 (Custom).jpg			10/03/2026 11:38:33	
7	7	user@example.com	mango_anthracnose	0.9856699109077454	20211011_133430 (Custom).jpg			10/03/2026 11:38:33	
8	8	user@example.com	mango_anthracnose	0.9856699109077454	20211011_133430 (Custom).jpg			10/03/2026 11:38:33	
9	9	user@example.com	mango_anthracnose	0.9856699109077454	20211011_133430 (Custom).jpg			10/03/2026 11:38:34	
10	10	user@example.com	mango_anthracnose	0.9856699109077454	20211011_133430 (Custom).jpg			10/03/2026 11:38:34	
11	11	user@example.com	mango_anthracnose	0.9993546605110168	20211008_125016 (Custom).jpg			10/03/2026 11:43:24	
12	12	user@example.com	potato_healthy	0.45796385407447815	20230712_121732.jpg			28/03/2026 14:28:53	
13	13	user@example.com	potato_healthy	0.45796385407447815	20230712_121732.jpg			28/03/2026 14:28:54	
14	14	user@example.com	mango_anthracnose	0.9993491768836975	20211011_134057 (Custom).jpg			28/03/2026 14:59:31	
15	15	user@example.com	mango_anthracnose	0.9993491768836975	20211011_134057 (Custom).jpg			28/03/2026 14:59:31	
16	16	user@example.com	mango_anthracnose	0.9993491768836975	20211011_134057 (Custom).jpg			28/03/2026 15:23:04	
17	17	user@example.com	mango_anthracnose	0.9901906251907349	20211008_125102 (Custom).jpg			28/03/2026 15:23:04	
18	18	user@example.com	chili_mites_and_trips	0.7324060797691345	mites_trips_1_2406.jpg			28/03/2026 15:23:16	
19	19	user@example.com	mango_die_back	0.9999926090240479	20211129_162341 (Custom).jpg			28/03/2026 17:06:29	
20	20	user@example.com	mango_die_back	0.9999926090240479	20211129_162341 (Custom).jpg			28/03/2026 17:06:29	
21	21	user@example.com	mango_die_back	0.9999926090240479	20211129_162341 (Custom).jpg			28/03/2026 17:06:29	
22	22	user@example.com	mango_bacterial_canker	0.8018844723701477	IMG_20211106_121349 (Custom).jpg			28/03/2026 17:06:30	

Figure 5.7: Database Table

Chapter 6: Summary and Conclusions

6.1 Summary

The Smart Food Ingredient Analyzer and Nutrition Awareness Web Application is a web-based system developed to help users understand the contents of packaged food products in a simple and user-friendly manner. The application allows users to upload food label images, from which text is extracted using Optical Character Recognition (OCR). The extracted ingredients are then processed and analyzed using a Large Language Model (LLM) to generate meaningful insights.

The system provides simplified ingredient explanations, a health score, possible health effects, daily consumption recommendations, and healthier alternative suggestions. Additionally, a comparison feature enables users to analyze two products side-by-side and identify the better option.

The application integrates modern technologies such as React for frontend development, Node.js and Express for backend processing, MongoDB for data storage, and external services like Google Vision API and OpenAI API for intelligent data processing and analysis.

6.2 Conclusion

This project successfully demonstrates how artificial intelligence and modern web technologies can be combined to solve real-world problems in the food and health domain. The system simplifies complex ingredient information into easy-to-understand insights, making it accessible to users without technical or scientific backgrounds.

By incorporating OCR, AI-based analysis, and caching mechanisms, the application ensures efficiency, scalability, and responsiveness. The product comparison feature further enhances user decision-making by enabling quick evaluation of multiple products.

Overall, the Smart Food Ingredient Analyzer provides a practical and effective solution for promoting healthier food choices and increasing awareness about food ingredients. It highlights the potential of AI-driven applications in improving everyday decision-making and public health awareness.

Chapter 7: Future Scope

While the current system provides a strong foundation, several enhancements can further improve its capabilities:

- Integration of a dedicated ingredient knowledge base for more consistent and explainable analysis
- Development of a hybrid scoring system combining rule-based logic with AI for improved accuracy
- Support for real-time scanning using a mobile camera
- Personalized recommendations based on user health profiles (e.g., allergies, diabetes)
- Multi-language support for wider accessibility
- Expansion into a mobile application (Android/iOS)
- Integration with barcode scanning for faster product identification

These improvements can make the system more robust, scalable, and suitable for real-world deployment.

Appendix

The appendix includes supplementary materials that support the development and evaluation of the Smart Food Ingredient Analyzer system:

- Sample input food label images used during testing and validation
- Screenshots of system outputs, including ingredient analysis and comparison results
- Sample API request and response formats for the /api/upload endpoint
- Additional system design artifacts such as Data Flow Diagrams, Activity Diagrams, Flowcharts, and Sequence Diagrams

These materials provide deeper insight into the system's functionality and implementation details beyond the main report.

Bibliography

- Google Cloud Vision API Documentation. Available at: <https://cloud.google.com/vision>
- OpenAI API Documentation. Available at: <https://platform.openai.com/docs>
- MongoDB Documentation. Available at: <https://www.mongodb.com/docs>
- React.js Official Documentation. Available at: <https://react.dev>
- Node.js Documentation. Available at: <https://nodejs.org>
- Express.js Documentation. Available at: <https://expressjs.com>
- Mongoose Documentation. Available at: <https://mongoosejs.com>
- Cloudinary Documentation. Available at: <https://cloudinary.com/documentation>

Research Papers:

1. Kumar, A. P., Nayak, M. S., Manoj, R. J., & Priyadarshi, A. (2022). *Pattern-Based Syntactic Simplification of Compound and Complex Sentences*. IEEE Access.
2. Naseer, A., Tamoor, M., Allheib, N., & Kanwal, S. (2024). *Investigating the Taxonomy of Character Recognition Systems: A Systematic Literature Review*. IEEE Access.
3. Lei, Z., Yang, S., Liu, H., Aslam, S., Liu, J., Tekle, H., Bugingo, E., & Zhang, D. (2018) *Mining of Nutritional Ingredients in Food for Disease Analysis*. IEEE Access.